
UNIT 6 STACKS, QUEUES, AND DEQUES

Structure

Page Nos.

6.0	Introduction
6.1	Objectives
6.2	Abstract Data Type-List
6.3	Array Implementation of Lists
6.4	Linked Lists-Implementation
6.5	Doubly Linked Lists-Implementation
6.6	Circularly Linked Lists-Implementation
6.7	Applications
6.8	Summary
6.9	Solutions/Answers
6.10	Further Readings

6.0 INTRODUCTION

In computer science, data structures are essential for organizing and managing data efficiently. Among the various types of data structures, **stacks**, **queues**, and **double-ended queues (deque)** are fundamental concepts that enable developers to handle collections of data in specific ways.

Stacks operate on a Last In, First Out (LIFO) basis, where the most recently added element are removed first. This structure is analogous to a stack of plates; you can only add or remove the top plate. We use Stacks for expression evaluation, backtracking and for function call management.

Queues, on the other hand, follow a First In, First Out (FIFO) principle. This means first added element to the queue will be the first one to be removed, similar to a line of customers at a service counter. Queues are widely utilized in scheduling tasks, managing requests in web servers, and handling asynchronous data processing.

Double-ended queues (deque) provide the flexibility of insertion and removal from both ends. This allows for operations similar to both stacks and queues. Deques are particularly useful in scenarios where elements need to be processed from both ends efficiently, such as in certain algorithms for breadth-first search or when implementing undo mechanisms in applications. Understanding these data structures is crucial for effective programming and algorithm design. They provide the foundational tools needed to manipulate collections of data efficiently and can significantly impact the performance of applications.

6.1 OBJECTIVES

After completing this unit, you will be able to:

- Define and differentiate between stacks, queues, and dequeues
- Implement basic operations for each data structure in Python
- Analyse the time complexity of operations in these structures

- Choose the best data structure for specific use cases
- Apply these data structures to solve real-world problems
- Understand and implement common variations of these structures

6.2 INTRODUCTION TO STACKS

A stack is one of the most fundamental abstract data structures in computer programming. We add and remove the elements from the same end of the stack which follows Last-in-first-out (LIFO) principle. Understanding stacks is crucial for many programming applications, from managing program execution to implementing undo operations in applications.

This chapter explores the implementation, operations, and practical applications of stacks in Python, providing you with the knowledge to effectively use this important data structure in your programming projects.

Understanding Stacks

A stack can be visualized as a vertical list of elements. The operations associated with stacks are primarily:

- **Push:** Adding an element to the top of the stack.
- **Pop:** Removing the element from the top of the stack.
- **Peek:** Retrieving the top element without removing it.

Below given diagram shows the push and pop operations

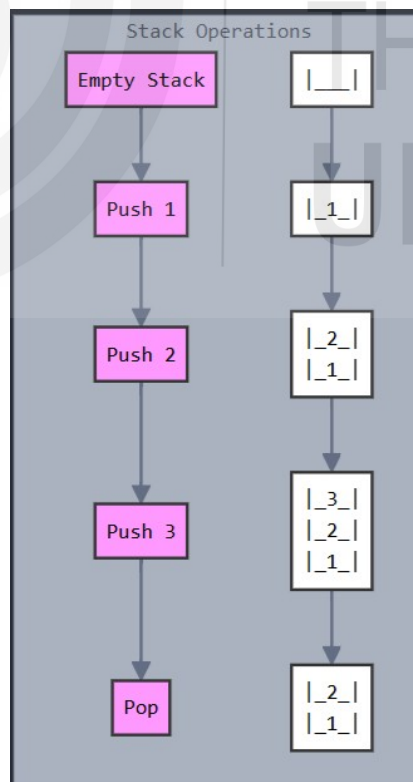


Figure 6.1 : Stack Operations

Characteristics of Stacks

- **LIFO Principle:** The last element added is removed first
- **Dynamic Size:** Stacks grow when we add or delete elements.
- **Limited Access:** Elements can only be accessed from stack's top.

Implementing a Stack in Python

In Python, stacks can be easily implemented using lists or the collections.deque module. Below are examples of both implementations.

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        raise IndexError("pop from empty stack")

    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        raise IndexError("peek from empty stack")

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

# Example usage
stack = Stack()
stack.push(1)
stack.push(2)
print(stack.peek()) # Output: 2
print(stack.pop())  # Output: 2
print(stack.size()) # Output: 1
```

Using collections.deque

The deque class from the collections module provides an efficient way to implement stacks due to its optimized performance for appending and popping elements.

```
from collections import deque
```

```
class Stack:
```

```
    def __init__(self):
```

```
        self.items = deque()
```

```
    def push(self, item):
```

```
        self.items.append(item)
```

```
    def pop(self):
```

```
        if not self.is_empty():
```

```
            return self.items.pop()
```

```
        raise IndexError("pop from empty stack")
```

```
    def peek(self):
```

```
        if not self.is_empty():
```

```
            return self.items[-1]
```

```
        raise IndexError("peek from empty stack")
```

```
    def is_empty(self):
```

```
        return len(self.items) == 0
```

```
    def size(self):
```

```
        return len(self.items)
```

```
# Example usage
```

```
stack = Stack()
```

```
stack.push(3)
```

```
stack.push(4)
```

```
print(stack.peek()) # Output: 4
```

```
print(stack.pop()) # Output: 4
```

```
print(stack.size()) # Output: 1
```

Key Operations

1. Push Operation:

- Adds an element to the top of the stack.
- Time Complexity: $O(1)$

2. Pop Operation:

- Removes and returns the top element of the stack.
- Time Complexity: $O(1)$

3. Peek Operation:

- Returns the top element without removing it.
- Time Complexity: $O(1)$

4. Is Empty Check:

- Checks if the stack is empty.
- Time Complexity: $O(1)$

5. Size Check:

- Returns the number of elements in the stack.
- Time Complexity: $O(1)$

Performance Considerations

Different stack implementations offer varying performance characteristics:

Operation	List based Stack	Deque-based stack
Push	$O(1)$	$O(1)$
Pop	$O(1)$	$O(1)$
peek	$O(1)$	$O(1)$
size	$O(1)$	$O(1)$

Best Practices

- Use collections.deque for better performance when implementing stacks due to its optimized append and pop operations.
- Always check for underflow conditions (attempting to pop from an empty stack) to avoid runtime errors.
- Consider encapsulating stack operations within a class to maintain data integrity and provide a clear interface.

Practical Use Cases

Stacks have numerous applications in computer science and programming:

- **Function Call Management:** We keep track of active functions and program execution variables in the call stack
- **Expression Evaluation:** Stacks are used in parsing expressions (e.g., infix to postfix expressions).

- **Backtracking Algorithms:** Algorithms like depth-first search (DFS) utilize stacks to explore nodes in graphs or trees efficiently.
- **Undo Mechanisms:** The word editors that handle undo operations use stack for managing the earlier states.

Understanding stacks is essential for effective programming and algorithm design. Stacks are used in various use cases. By mastering stack operations and their implementations in Python, developers can leverage this powerful data structure in their coding

6.3 QUEUES

In Python programming, queues store and manipulate elements in a FIFO (First-In-First-Out) manner. Unlike static data structures, queues provide dynamic and flexible mechanisms for managing ordered collections where the first element is the first one to be removed. Queues are essential in various programming scenarios, particularly in managing tasks, scheduling processes, and handling asynchronous data.

In this section, we will discuss the queue data structure in Python, covering its implementation, key operations, best practices, and practical use cases.

Understanding Queues

A queue can be visualized as a collection of elements arranged in a linear order. The operations associated with queues include:

- **Enqueue:** Adds an element to the back of the queue.
- **Dequeue:** Removing an element from the queue front
- **Peek:** Retrieving the front element without removing it.
- **IsEmpty:** Checks the emptiness of the queue
- **Size:** Returns the number of elements in the queue

Below given diagram shows Enqueue and Dequeue operations –

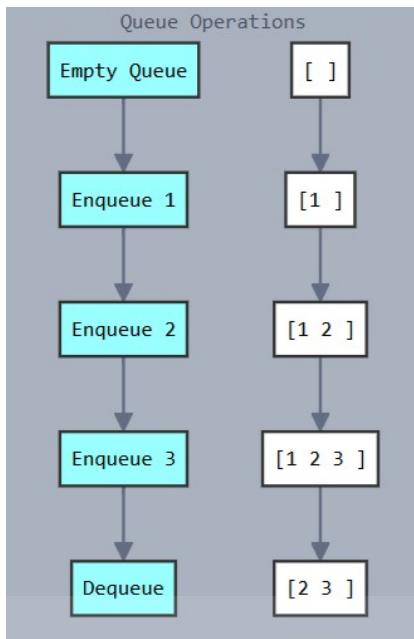


Figure 6.2 : Queue Operations

Fundamental Queue Types

Python offers several ways to implement queues:

1. **List-based Queues:** Simple way to implement the queues.
2. **collections.deque:** Optimized for queue operations
3. **queue.Queue:** Thread-safe implementation for concurrent programming
4. **Priority Queues:** Special queues where elements have priorities

Characteristics of Queues

- FIFO Principle: The first element added is the first to be removed.
- Dynamic Size: Queues can grow or shrink as elements are added or removed.
- Limited Access: Elements can only be accessed from the queue front.

Implementing a Queue in Python

In Python, queues can be implemented using lists or the 'collections.deque' module. Below are examples of both implementations.

Using Lists

```

class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        raise IndexError("dequeue from empty queue")

```

```

def peek(self):
    if not self.is_empty():
        return self.items[0]
    raise IndexError("peek from empty queue")

def is_empty(self):
    return len(self.items) == 0

def size(self):
    return len(self.items)

# Example usage
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
print(queue.peek()) # Output: 1
print(queue.dequeue()) # Output: 1
print(queue.size()) # Output: 1
'''

```

Using `collections.deque`

The `deque` class from the `collections` module provides an efficient way to implement queues due to its optimized performance for appending and popping elements from both ends.

```

from collections import deque

class Queue:
    def __init__(self):
        self.items = deque()

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.popleft()
        raise IndexError("dequeue from empty queue")

    def peek(self):
        if not self.is_empty():
            return self.items[0]
        raise IndexError("peek from empty queue")

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)

```

Example usage

```

queue = Queue()
queue.enqueue(3)
queue.enqueue(4)
print(queue.peek()) # Output: 3
print(queue.dequeue()) # Output: 3
print(queue.size()) # Output: 1
'''

```

Key Operations

1. Enqueue Operation:

- Adds an element to the back of the queue.
- Time Complexity: $O(1)$ when using `collections.deque`, $O(n)$ when using lists due to shifting elements.

2. Dequeue Operation:

- Removes and returns the front element.
- Time Complexity: $O(1)$ when using `collections.deque`, $O(n)$ when using lists.

3. Peek Operation:

- Returns the front element without removing it.
- Time Complexity: $O(1)$

4. Is Empty Check:

- Checks if the queue is empty.
- Time Complexity: $O(1)$

5. Size Check:

- Returns the number of elements in the queue.
- Time Complexity: $O(1)$

Performance Characteristics

Given below are performance characteristics of various queue implementations–

Operation	List-based Queue	deque-based Queue	Queue-based Queue	Priority Queue
Enqueue	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$
Dequeue	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$
Front	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Size	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Best Practices

Given below are the main best practices:

1. Choose the Right Implementation

- Use `collections.deque` for general-purpose queues
- Use `Queue.Queue` for thread-safe operations

- Use PriorityQueue when elements have priorities
- 2. **Exception Handling**
 - Always handle queue underflow and overflow conditions
 - Implement proper error messages for exceptional cases
- 3. **Memory Management**
 - Consider implementing maximum size limits
 - Clear queues when no longer needed
- 4. **Performance Optimization**
 - Use deque for better performance in large queues
 - Avoid list-based implementation for large datasets

Practical Use Cases

Queues have numerous applications in computer science and programming:

- Task Scheduling: Operating systems use queues to manage processes and scheduling tasks efficiently.
- Breadth-First Search (BFS): Queues are used in in BFS based graph traversal to explore nodes level by level.
- Print Queue Management: Printers manage print jobs using queues to ensure that documents are printed in the order they were received.
- Asynchronous Data Handling: Queues handle communication across multithreaded environments.

Let us look at the code for queues for task scheduling as given below -

```
class TaskScheduler:
    def __init__(self):
        self.task_queue = Queue()

    def add_task(self, task):
        self.task_queue.enqueue(task)

    def process_tasks(self):
        while not self.task_queue.is_empty():
            task = self.task_queue.dequeue()
            print(f"Processing task: {task}")
            # Execute task logic here

# Example usage
scheduler = TaskScheduler()
scheduler.add_task("Send email")
scheduler.add_task("Generate report")
scheduler.add_task("Backup database")
scheduler.process_tasks()
```

Another example of using Queue for print job as given below -

```
class PrintQueue:
    def __init__(self):
        self.print_queue = PriorityQueue()
```

```
def add_document(self, document, priority):
    self.print_queue.enqueue(document, priority)

def process_print_jobs(self):
    while not self.print_queue.is_empty():
        document = self.print_queue.dequeue()
        print(f"Printing: {document}")
```

Understanding queues is crucial for effective programming and algorithm design. By mastering queue operations and their implementations in Python, developers can leverage this powerful data structure in their coding practices effectively.

6.4 DOUBLE ENDED QUEUES

A double-ended queue, commonly referred to as a deque, allows insertion and removal of elements from both ends, making it a hybrid between stacks and queues. This flexibility enables efficient operations at both the front and the back of the structure, making deques suitable for a variety of applications in programming.

In this section, we will explore the deque data structure in Python, including its implementation, key operations, best practices, and practical use cases.

Understanding Deques

A deque can be visualized as a linear collection of elements where you can add or remove items from both ends. The primary operations associated with deques include:

- Append: Adding an element to the back of the deque.
- Appendleft: Adding an element to the front of the deque.
- Pop: Removing an element from the back of the deque.
- Popleft: Removing an element from the front of the deque.
- Peek: Retrieving the front or back element without removing it.

Below given diagram shows append, peek, pop operations in dequeue -

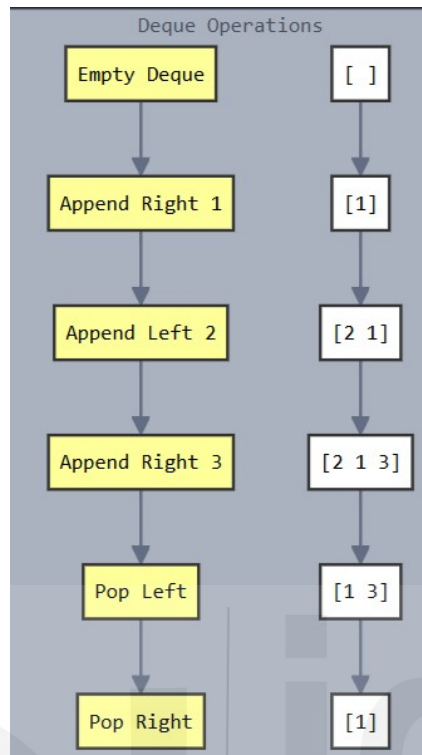


Figure 6.3 : Deque Operations

Characteristics of Deques

- **Double-Ended:** Elements can be added or removed from either end
- **Thread-Safe:** Safe for concurrent access from multiple threads
- **Memory-Efficient:** Optimized memory usage compared to lists
- **Fast Operations:** $O(1)$ complexity for append and pop operations at both ends
- **Flexible Operations:** Elements can be added or removed from both ends.
- **Dynamic Size:** Deques can grow or shrink as elements are added or removed.
- **Efficient Access:** Fast access to both ends with constant time complexity for pop and append operations.

Implementing a Deque in Python

In Python, `'collections.deque'` class is used for implementing deque, which provides an efficient way to manage double-ended queues.

```
# Using 'collections.deque'
```

```
from collections import deque
```

```
class Deque:
    def __init__(self):
        self.items = deque()
```

```
def append(self, item):
    self.items.append(item)
```

```
def appendleft(self, item):
    self.items.appendleft(item)
```

```
def pop(self):
    if not self.is_empty():
        return self.items.pop()
    raise IndexError("pop from empty deque")
```

```
def popleft(self):
    if not self.is_empty():
        return self.items.popleft()
    raise IndexError("popleft from empty deque")
```

```
def peek(self):
    if not self.is_empty():
        return self.items[0]
    raise IndexError("peek from empty deque")
```

```
def is_empty(self):
    return len(self.items) == 0
```

```
def size(self):
    return len(self.items)
```

```
# Example usage
deque = Deque()
deque.append(1)
deque.append(2)
deque.appendleft(0)
print(deque.peek()) # Output: 0
print(deque.pop())  # Output: 2
print(deque.popleft()) # Output: 0
print(deque.size()) # Output: 1
'''
```

Key Operations

1. Append Operation:

- Adds an element to the back of the deque.
- Time Complexity: $O(1)$

Adding single elements

```
my_deque = deque()
```

```
my_deque.append(10)    # Adds to right
```

```
my_deque.appendleft(5) # Adds to left
```

Adding multiple elements

```
my_deque.extend([20, 30]) # Extends on right
```

```
my_deque.extendleft([3, 2, 1]) # Extends on left (note: elements are added in reverse)
```

2. Appendleft Operation:

- Adds an element to the front of the deque.
- Time Complexity: $O(1)$

3. Pop Operation:

- Removes the back element of the deque.
- Time Complexity: $O(1)$

Basic removal

```
right_val = my_deque.pop()
```

```
left_val = my_deque.popleft()
```

Clearing the deque

```
my_deque.clear()
```

4. Popleft Operation:

- Removes the front element of the deque.
- Time Complexity: $O(1)$

5. Peek Operation:

- Returns the front element without removing it.
- Time Complexity: $O(1)$

6. Is Empty Check:

- Checks if the deque is empty.
- Time Complexity: $O(1)$

7. Size Check:

- Returns the deque element size.
- Time Complexity: $O(1)$

Performance Characteristics

Operation	Deque	List
Append/Pop at end	$O(1)$	$O(1)$
Append/Pop at beginning	$O(1)$	$O(n)$
Random Access	$O(n)$	$O(1)$
Length Check	$O(1)$	$O(1)$
Memory Usage	Lower	Higher

Best Practices

- Utilize `'collections.deque'` for implementing double-ended queues due to its optimized performance for appending and popping elements at both ends.
- Always check for underflow conditions (attempting to pop or popleft from an empty deque) to avoid runtime errors.
- Encapsulate deque operations within a class to maintain data integrity and provide a clear interface.

When to use Dequeues

- Use deques when you need:
 - Fast append/pop operations at both ends
 - Thread-safe operations
 - Memory-efficient storage
 - Rotation operations
- Prefer lists when you need:
 - Random access to elements
 - Slice operations
 - Memory is not a constraint

Practical Use Cases

Deques have numerous applications in computer science and programming:

- Task Scheduling: Deques can efficiently manage tasks that require processing from both ends, such as in breadth-first search algorithms where nodes are added and removed dynamically.
- Sliding Window Problems: Deques are often used in algorithms that require maintaining a window of elements for computations like maximum or minimum values over a sliding range.

- Undo Mechanisms: Applications such as text editors can utilize deques to manage undo and redo operations by keeping track of changes at both ends.
- Buffer Management: Deques can serve as buffers that handle streaming data efficiently by allowing insertion and removal of data packets at both ends.

Sample code for sliding window

```
from collections import deque
```

```
def sliding_window_max(nums, k):
    """Find maximum elements in sliding windows of size k."""
    result = []
    window = deque()

    for i, num in enumerate(nums):
        # Remove elements outside current window
        while window and window[0] < i - k + 1:
            window.popleft()

        # Remove smaller elements
        while window and nums[window[-1]] < num:
            window.pop()

        window.append(i)

        if i >= k - 1:
            result.append(nums[window[0]])

    return result

# Example usage
nums = [1, 3, -1, -3, 5, 3, 6, 7]

k = 3
print(sliding_window_max(nums, k)) # Output: [3, 3, 5, 5, 6, 7]
```

Sample code for history tracking

```
class BrowserHistory:
    def __init__(self):
        self.history = deque()
        self.current_index = -1

    def visit(self, url):
```

```

    # Remove forward history
    while len(self.history) > self.current_index + 1:
self.history.pop()
self.history.append(url)
self.current_index += 1

    def back(self, steps):
        steps = min(steps, self.current_index)
self.current_index -= steps
        return self.history[self.current_index]

    def forward(self, steps):
remaining_steps = len(self.history) - self.current_index - 1
        steps = min(steps, remaining_steps)
self.current_index += steps
        return self.history[self.current_index]

```

Implementation Guidelines

1. Size Management

```

# Creating a deque with maximum size
limited_deque = deque(maxlen=3)
for i in range(5):
    limited_deque.append(i)
print(limited_deque) # Output: deque([2, 3, 4], maxlen=3)

```

2. Thread Safety

```

from collections import deque
from threading import Thread

```

```

shared_deque = deque()

```

```

def worker():
    for i in range(1000):
shared_deque.append(i)
        if shared_deque:
shared_deque.pop()

```

```

threads = [Thread(target=worker) for _ in range(4)]
for t in threads:
    t.start()

```

Understanding double-ended queues is essential for effective programming and algorithm design. Their flexibility in managing elements at both ends can be used for implementing various applications across different domains. By mastering deque operations and their implementations in Python, developers can leverage this powerful data structure in their coding practices effectively.

Check your progress 1

1. What principle does a stack follow? a) FIFO b) LIFO c) Random Access d) Sequential Access

2. In a queue, if we remove an element, from which end does it happen? a) Front b) Rear c) Middle d) Any position
3. Which operation would you use to add an element to a stack? a) enqueue b) push c) insert d) add
4. A dequeue allows operations on: a) Only front end b) Only rear end c) Both ends d) Middle only
5. What is the time complexity of push operation in a stack? a) $O(n)$ b) $O(1)$ c) $O(\log n)$ d) $O(n^2)$
6. True or False: A queue can be implemented using two stacks.
7. Which of these applications would best use a stack? a) Print queue in an operating system b) Undo operation in a text editor c) Breadth-first search in a graph d) Round-robin scheduling
8. What happens when you try to pop from an empty stack? a) Returns null b) Throws an exception c) Returns -1 d) Depends on implementation.

6.5 SOLUTIONS/ANSWERS TO CHECK YOUR PROGRESS

1. Answer: b) LIFO
2. Answer: a) Front
3. Answer: b) push
4. Answer: c) Both ends
5. Answer: b) $O(1)$
6. Answer: True
7. Answer: b) Undo operation in a text editor
8. Answer: b) Throws an exception

6.6 SUMMARY

Stacks, queues, and dequeues are fundamental data structures that are used in computer programming:

Stacks (LIFO):

- Support push and pop operations
- Ideal for tracking state and handling recursion
- Common applications include expression evaluation and backtracking
- $O(1)$ is the basic operations time complexity

Queues (FIFO):

- Support enqueue and dequeue operations
- Perfect for scheduling and buffering
- Used in breadth-first search and print spooling
- $O(1)$ is the basic operations time complexity

Dequeues:

- Combine features of stacks and queues
- Allow insertion and deletion at both ends

- Useful for sliding window problems and palindrome checking
- Provide flexibility with same performance benefits

Key Implementation Considerations:

- Can be implemented using arrays or linked lists
- Array implementations may require resizing
- Linked list implementations offer dynamic memory usage
- Choice depends on specific use case requirements

These data structures are the basic foundations for more complex algorithms and systems, making them essential tools in any programmer's toolkit.

6.7 FURTHER READINGS

1. Books:

- **"Data Structures and Algorithms in Python" by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser:** This book covers fundamental data structures, including stacks, queues, and deques, with a focus on implementation in Python.
- **"Python Data Structures and Algorithms" by Benjamin Baka:** This book offers a practical approach to implementing various data structures in Python, including detailed sections on stacks and queues.

2. Online Courses:

- **Coursera - "Data Structures and Algorithms Specialization":** This course series includes modules specifically focused on stacks and queues, providing both theoretical knowledge and practical coding exercises.
- **edX - "Introduction to Computer Science using Python":** This course covers basic data structures including stacks and queues with hands-on programming assignments.

3. Documentation and Tutorials:

- **Python Official Documentation:** The official Python documentation provides insights into the collections module where deques are implemented. It includes usage examples and performance considerations.
- **GeeksforGeeks - Data Structures:** This website offers tutorials on various data structures including stacks, queues, and deques with examples in Python.
- **Real Python - "Understanding Deque in Python":** A detailed article that explains the deque data structure with examples of its usage in real-world applications.